

PRACTICAL CASE STUDY

E-Commerce Sales Analytics using DuckDB

Python, SQL, DuckDB, Parquet and Streamlit

A reproducible analytical platform that cleans a 250,000-row source, models a star schema, creates OLAP queries, benchmarks storage formats, and serves a decision-focused dashboard.

Student / Group	_____
Roll number(s)	_____
Course	Data Engineering
Institution	SIES Graduate School of Technology
Submission date	_____

1 Executive summary

Problem statement, scope and measurable outcome

E-commerce decision makers need fast, reusable answers about revenue, growth, product performance, regional demand, customer retention and returns. Raw CSV files are easy to exchange but expensive to scan repeatedly and commonly contain inconsistent text, missing values and invalid types.

This implementation builds a local analytical platform around DuckDB. Python generates or accepts CSV input. SQL performs type-safe cleaning, dimensional modeling and reusable metrics. The pipeline exports partitioned Parquet, records query plans and measures performance across storage and thread modes. Streamlit provides an interactive presentation layer.

248,555 clean fact rows	\$28.78M realized revenue	119,044 distinct orders	77.51% repeat customers
-----------------------------------	-------------------------------------	-----------------------------------	-----------------------------------

Key result

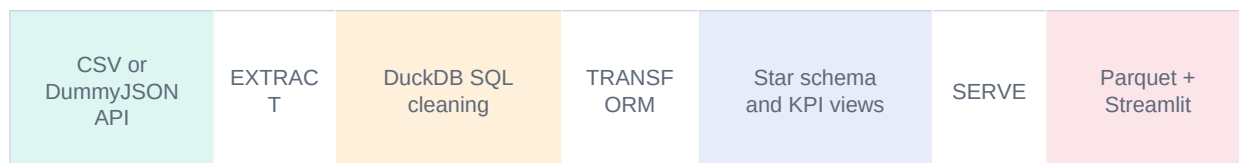
The benchmark query completed in **29.319 ms** on a four-thread DuckDB table versus **674.031 ms** on single-thread CSV, a measured **23.0x speedup**. Results depend on this machine, dataset and query, so the repository includes the benchmark script for reproduction.

Project objectives

Objective	Implemented evidence
Load and clean CSV	TRY_CAST, range rules, duplicate removal, rejection reasons
Model for analytics	Fact table plus customer, product and date dimensions
Apply OLAP SQL	Views, joins, LAG, LEAD, DENSE_RANK, ROLLUP, CUBE, GROUPING SETS
Optimize	Partitioned ZSTD Parquet, indexes, thread settings, EXPLAIN ANALYZE
Communicate insights	Five dashboard views plus documented findings

2 System architecture and workflow

A local OLAP design with a deployable dashboard



Pipeline sequence

Step	Action	Persistent output
1	Generate a deterministic source or ingest a public API snapshot	CSV and raw JSON
2	Read all CSV columns as text to prevent ingestion failure	warehouse.raw_orders
3	Standardize values, type with TRY_CAST, deduplicate and quarantine failed orders and rejected_orders	warehouse.raw_orders
4	Build dimensions and fact table, then define KPI views	ecommerce.duckdb
5	Export analytics-ready rows by year and month	partitioned ZSTD Parquet
6	Run queries, plans, charts and benchmark repetitions	results folder
7	Serve the warehouse through Streamlit or a Kubernetes Service	interactive dashboard

Design choice

DuckDB suits a classroom practical because it provides vectorized OLAP execution, parallel scans, direct CSV and Parquet access, window functions and multidimensional grouping without operating a distributed cluster. The same SQL-first habits transfer to larger warehouse systems.

Core tools

Layer	Technology	Role
Ingestion	Python standard library and pandas	Synthetic generation and current API snapshot
Processing	DuckDB SQL	Cleaning, modeling, views, OLAP and plans
Storage	DuckDB database and Parquet	Fast repeated queries and portable columnar files
Presentation	Streamlit and Plotly	Interactive filters, KPIs and charts
Deployment	Docker and Kubernetes	Portable service with probes, PVC and HPA

3 Dataset and data quality

Deterministic source data with defects added for the cleaning exercise

250,250 raw rows	250,000 deduplicated rows	1,445 rejected rows	0.578% rejection rate
----------------------------	-------------------------------------	-------------------------------	---------------------------------

Source profile

Attribute	Value
Coverage	2024-01-01 through 2025-12-31
Business grain	One order line per row
Products	48 products in six categories
Customers	37,114 unique cleaned customers
Orders	119,044 distinct orders
Quality defects	Null customer IDs, null prices, negative quantities, excessive discounts, inconsistent casing and duplicates

Cleaning rules

Field or issue	Rule	Treatment
Duplicate line_id	ROW_NUMBER() over line_id	Keep one deterministic row
Dates and numerics	TRY_CAST	Invalid conversion becomes NULL
Customer and product IDs	Must be present	Reject row
Quantity	1 to 20	Reject outside range
Unit price	Greater than zero	Reject missing or non-positive
Discount	0 to 80 percent	Reject outside range
Region, category, status	Trim and normalize case	Map to canonical value

Auditability

The pipeline keeps rejected rows with a semicolon-separated reason instead of silently discarding them. The final quality JSON records raw, typed, clean and rejected counts plus file sizes and build time. This supports reproducibility and makes the cleaning outcome defensible during evaluation.

4 Warehouse schema and reusable SQL

A compact star schema separates descriptive attributes from measurable events



Reusable views

View	Purpose
vw_sales_enriched	Joins fact and dimensions once for analyst-friendly access
vw_daily_revenue	Daily realized revenue, orders and customers
vw_monthly_growth	Monthly revenue plus LAG-based percentage growth
vw_repeat_customer_pct	Customers with more than one fulfilled order

Representative window query

```
WITH customer_revenue AS (  
  SELECT region, customer_id, SUM(net_revenue) AS revenue  
  FROM warehouse.vw_sales_enriched  
  WHERE order_status IN ('Delivered', 'Shipped')  
  GROUP BY region, customer_id  
)  
SELECT region, customer_id, revenue,  
       DENSE_RANK() OVER (  
         PARTITION BY region ORDER BY revenue DESC  
       ) AS regional_rank  
FROM customer_revenue  
QUALIFY regional_rank <= 5;
```

5 OLAP operations and query planning

Multidimensional totals answer related questions in one SQL statement

The project includes separate examples of ROLLUP, CUBE and GROUPING SETS. These operations calculate detailed groups, subtotals and the grand total while preserving a consistent measure definition.

Pattern	Dimensions	Decision question
ROLLUP	region, category	What are category results within each region, regional totals and the overall total?
CUBE	region, sales channel	How does each region-channel pair compare with every one-dimensional total?
GROUPING SETS	region-category, region, category, total	Can the dashboard receive only the aggregation levels it needs?

GROUPING SETS output

The selected grouping sets returned **42** result rows: 30 region-category details, five region totals, six category totals and one grand total.

Optimization evidence

Technique	Implementation	Expected effect
Columnar storage	ZSTD-compressed Parquet	Read only referenced columns and reduce disk size
Partition pruning	year and month folder partitions	Skip files outside a filtered period
Parallel execution	SET threads = 1 or 4	Use multiple cores when row groups allow it
Persistent database	DuckDB fact and dimensions	Avoid repeated CSV parsing and support repeated joins
Indexes	date, customer and product keys	Document access paths for selective lookup patterns
EXPLAIN ANALYZE	Saved plan text	Compare estimated work with actual operator timing

Storage result

The finished DuckDB file measured **22.012 MB**. The partitioned Parquet dataset measured **6.666 MB**. Parquet therefore used about **3.3x less disk space** in this run, while the database delivered the fastest repeated analytical query.

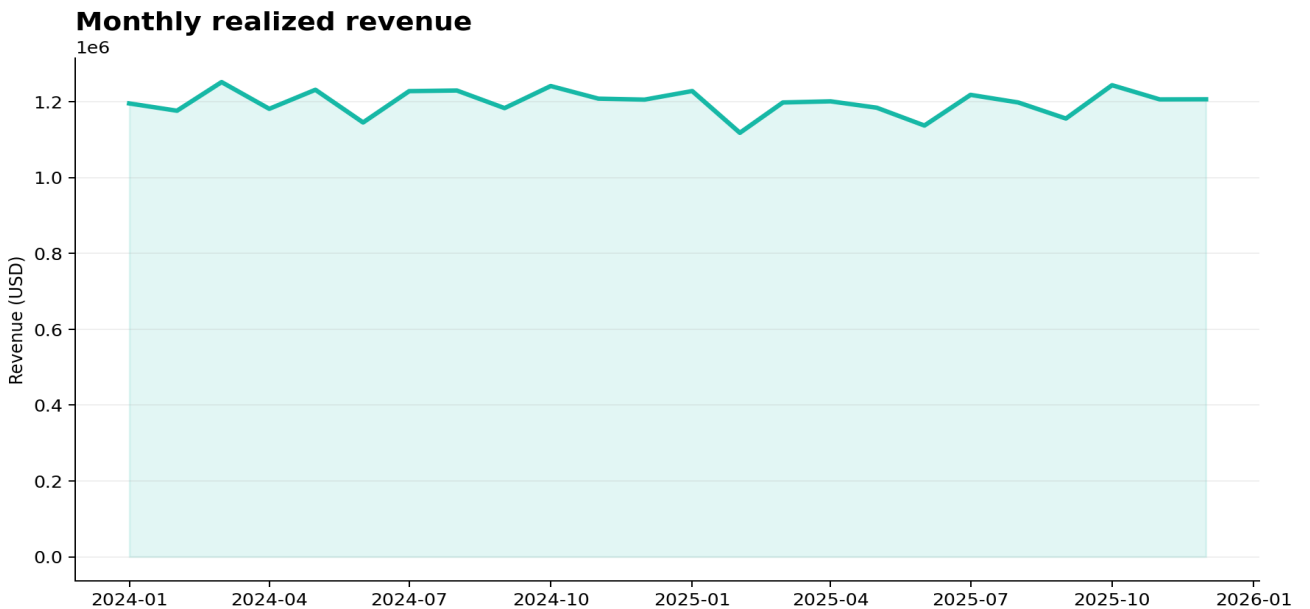
Interpretation

DuckDB documentation explains that parallelism depends on row groups and that too many threads can sometimes reduce performance. The benchmark therefore reports measured values instead of assuming that higher thread counts always win. The repository also preserves the query plan for review.

6 Business results

Measured findings from the cleaned warehouse

\$28.78M realized revenue	\$131.59 average fulfilled line	7.11% return rate	77.51% repeat customer rate
-------------------------------------	---	-----------------------------	---------------------------------------



The monthly series remains broadly stable across the two-year period, with visible month-to-month volatility. The reusable monthly-growth view provides the exact LAG-based percentage required to investigate each movement.

Decision implications

Finding	Evidence	Suggested decision
Electronics leads revenue	\$7.69M	Protect availability for top electronic products and test bundles
South is the largest region	\$6.51M	Use South as the baseline for channel and assortment comparisons
Sports has the highest return rate	7.38%	Review product descriptions, fit guidance and supplier quality
Repeat purchase is high	77.51%	Segment repeat customers and measure retention by acquisition channel

7 Category and regional analysis

Revenue concentration and return behavior guide operational priorities



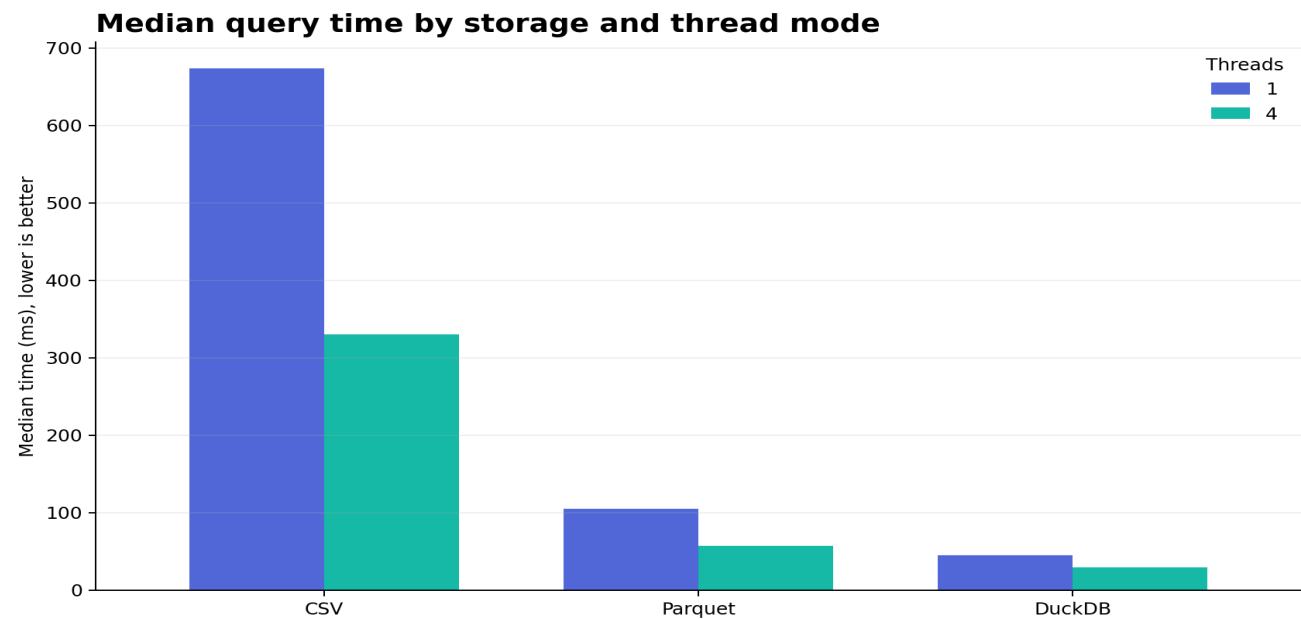
Category	Revenue	Orders	Return %
Electronics	\$7,690,726	37,527	6.93
Home	\$7,414,090	37,499	7.17
Fashion	\$4,777,188	37,513	7.16
Sports	\$3,653,106	37,496	7.38
Beauty	\$2,821,200	37,712	7.01
Books	\$2,420,729	37,521	6.98

Analytical caution

The source is synthetic and the API data is a public test service, so these findings demonstrate the analytical method rather than describe a real company. The implementation separates measures, dimensions and views so a real source can replace the generator without redesigning the dashboard.

8 Performance comparison

Five timed repetitions per storage and thread mode



Storage	Threads	Median ms	Min ms	Max ms
CSV	1	674.031	665.540	688.396
Parquet	1	104.650	100.067	118.481
DuckDB	1	44.775	42.669	48.432
CSV	4	330.050	323.678	333.697
Parquet	4	57.009	49.927	60.355
DuckDB	4	29.319	27.484	30.068

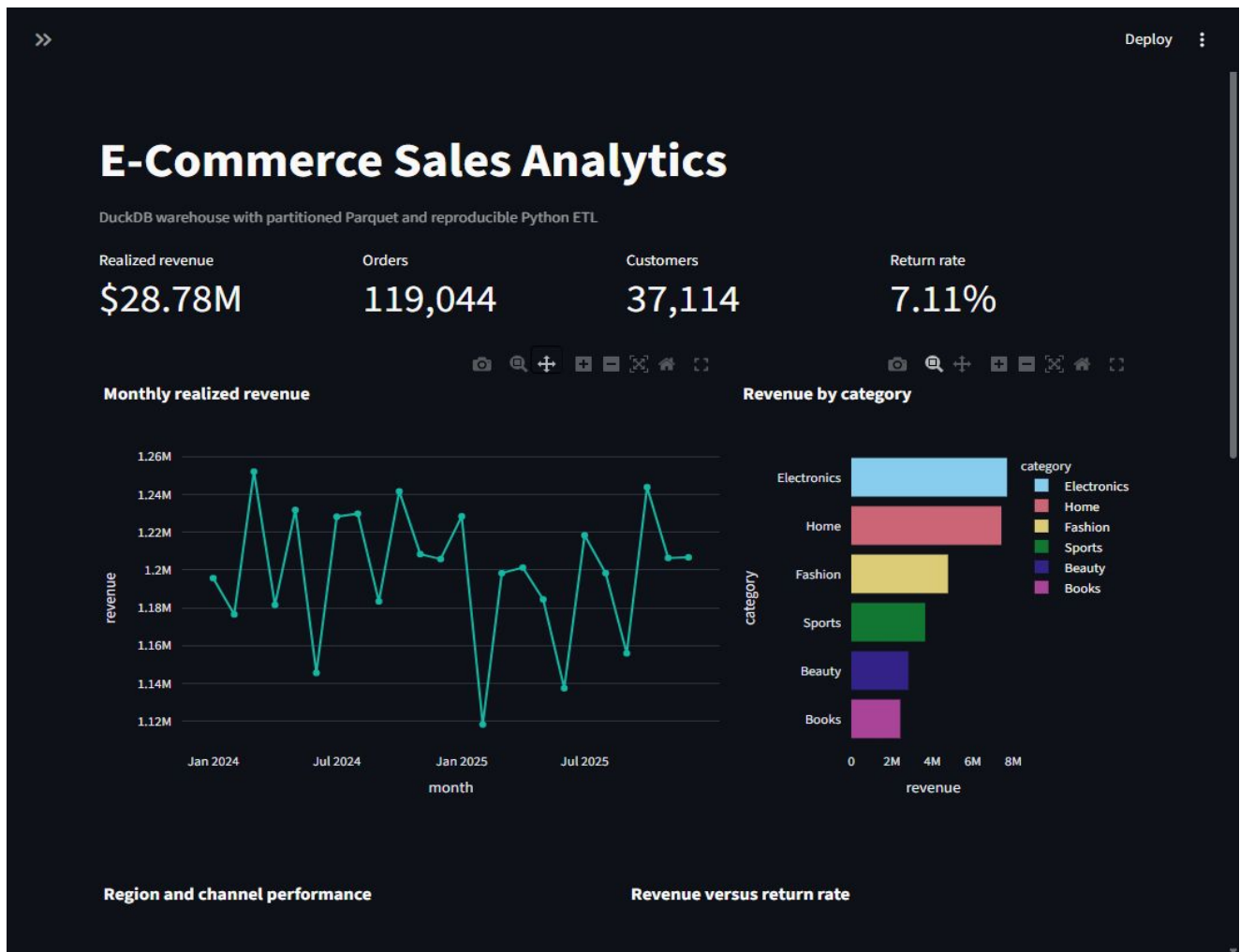
Measured interpretation

Comparison	Measured effect
CSV: 1 thread to 4 threads	2.04x faster
Parquet: 1 thread to 4 threads	1.84x faster
DuckDB: 1 thread to 4 threads	1.53x faster
CSV 1 thread versus DuckDB 4 threads	22.99x faster

The benchmark warms each query before timing and reports median, minimum and maximum values. It compares execution modes rather than claiming distributed cluster performance. This distinction is important because DuckDB runs inside a single process.

9 Interactive analytics dashboard

A live Streamlit interface over the DuckDB warehouse



Dashboard element	Question answered
Revenue, orders, customers and return KPIs	What is the current scale and quality of sales?
Monthly line chart	How does realized revenue change over time?
Category bar chart	Which categories generate the most revenue?
Region-channel grouped bars	Where does each sales channel perform best?
Revenue-return scatter	Which categories combine scale with return risk?
Top-products table	Which products deserve inventory and merchandising attention?

The sidebar provides date, region and category filters. Every query runs against the same cleaned view, which keeps KPI and chart definitions consistent.

10 Bonus implementation

Public API ingestion, Kubernetes deployment and execution-mode evidence

1. Current public API snapshot

The bonus ingestion calls the DummyJSON products and carts endpoints, records the UTC observation time, preserves the raw JSON, flattens nested cart items and loads the result into DuckDB. The final run captured **194 products**, **208 carts** and **800 cart lines** at **2026-10-08T18:42:14+00:00**.

2. Kubernetes deployment

Manifest	Purpose
Deployment	Runs the Streamlit container with CPU and memory requests/limits
Service	Exposes port 80 inside the cluster
PersistentVolumeClaim	Persists the DuckDB database and analytical artifacts
Startup, readiness and liveness probes	Use Streamlit's <code>/_stcore/health</code> endpoint
HorizontalPodAutoscaler	Scales from one to three replicas at 70% CPU, when Metrics Server is installed
Kustomization	Applies all namespaced resources with one command

3. Performance across execution modes

The benchmark crosses three storage choices with one-thread and four-thread settings. This creates six reproducible modes and directly tests storage parsing, columnar scans, persistent tables and multicore execution.

Deployment commands

```
docker build -t ecommerce-duckdb:1.0 .
kubectl apply -k kubernetes/
kubectl -n ecommerce-analytics port-forward svc/ecommerce-duckdb 8501:80
```

Scaling caveat

A single-writer DuckDB file on a ReadWriteOnce volume should not be treated as a horizontally writable distributed database. The included HPA demonstrates dashboard pod scaling, while production multi-replica designs should use read-only database snapshots, object-storage Parquet, or an external database service.

11 Validation, limitations and conclusion

What was verified and how the project can be extended

Validation performed

Check	Result
End-to-end smoke test	Passed with generated source, warehouse build and KPI view checks
Full evidence run	Processed 250,250 raw rows into 248,555 clean fact rows
API ingestion	Loaded 800 flattened API lines into warehouse.api_cart_items
Dashboard	Opened locally, rendered KPIs and charts, and captured visual evidence
SQL plans	Saved EXPLAIN ANALYZE output for category and benchmark queries
Artifact review	Report and slides rendered to images for layout inspection

Limitations

The primary dataset is synthetic, so monetary findings are illustrative. API carts come from a public test service rather than a transactional production system. The project provides Kubernetes manifests but does not prove a live multi-node cluster unless the evaluator runs them in Minikube, kind or another Kubernetes environment. Benchmark results are machine-specific and should be rerun on the evaluation computer.

Conclusion

The project demonstrates a complete analytical path from imperfect CSV to cleaned dimensions, fact records, reusable views, OLAP queries, performance evidence and an interactive dashboard. DuckDB reduced the benchmark median from hundreds of milliseconds for CSV parsing to tens of milliseconds for the persistent analytical table. The bonus API and deployment assets extend the work beyond a static classroom notebook.

Suggested extensions

Extension	Value
Incremental ingestion	Track a high-water mark and merge only new orders
Data expectations	Automate null, uniqueness, range and freshness checks
Object storage	Read partitioned Parquet from S3-compatible storage
Orchestration	Schedule generation, cleaning, validation and publication with Prefect
Real business data	Map an approved transaction export into the documented raw contract

12 References and viva preparation

Primary documentation used for implementation decisions

1. DuckDB. EXPLAIN ANALYZE: Profile Queries. https://duckdb.org/docs/current/guides/meta/explain_analyze
2. DuckDB. Tuning Workloads. https://duckdb.org/docs/current/guides/performance/how_to_tune_workloads
3. DuckDB. File Formats and Parquet performance guidance. https://duckdb.org/docs/current/guides/performance/file_formats
4. DuckDB. Parquet Tips. <https://duckdb.org/docs/current/data/parquet/tips>
5. DummyJSON. Products API. <https://dummyjson.com/docs/products>
6. DummyJSON. Carts API. <https://dummyjson.com/docs/carts>
7. Kubernetes. Horizontal Pod Autoscaling. <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>

Likely viva questions

Question	Short answer
Why DuckDB?	It provides embedded vectorized OLAP, SQL, parallelism and direct CSV/Parquet access without a separate
Why read the CSV as text first?	A malformed value cannot abort ingestion; TRY_CAST makes invalid values visible as NULL for explicit rules
Why use a star schema?	It gives one measurable fact grain and reusable descriptive dimensions for consistent aggregation.
Why was DuckDB faster than CSV?	The persistent table avoids repeated parsing and uses typed columnar execution.
What does partitioning improve?	Date filters can skip unrelated year-month files, reducing scanned data.
Is this distributed?	No. DuckDB is in-process. The project demonstrates distributed-style columnar and parallel processing on or
What does the HPA scale?	Dashboard pods. A writable DuckDB file still requires single-writer design discipline.

Repository handoff

Run `python -m src.cli all --rows 250000`, then `streamlit run src/dashboard.py`. The README maps every deliverable and bonus criterion to the corresponding source files.